How to HFT

yenwod (@yenwod_), November 9, 2026
https://x.com/yenwod_/status/2119896129794126320

Steps to build a minimal HFT trading stack and deploy it on decentralized crypto exchanges.
I expect this would take a couple months for a strong programmer new to trading. It's unlikely to be immediately remunerative, but if you like programming and puzzles it will be fun.
1. Write a program to connect to Hyperliquid and log all traffic at a level above packet capture - every TCP connection opened and closed, HTTP request and response, WS message out and in, etc. with the local timestamp. Write incoming events as soon as you see them, and outgoing events before you send them, by appending to a file. Run all connections on one thread. Use third party libraries only for the "sans-io" parts of this stack (websocket framing, TLS, etc). Do not use async language features.
2. Write a state machine that tracks the book and connection state. Write a main method that runs the Hyperliquid connection + logging code on one thread, and polls the file on another thread, taking events as the file grows and calling `sm.update(event)`. Make your state machine `update` print the timestamp and mid price and run the program.
3. Update the program to add a "replay" mode where it runs the file polling but not the networking thread. Run the program again and make sure it prints the exact same output. If it doesn't, fix the state machine until it is deterministic (eg get "current time" from the event, not from the system clock).
4. Allow the state machine to act. Make `sm.update` return one of (place, cancel, do nothing). Send actions to the networking thread and log actions to the same file before carrying them out (or ignore them in replay mode).
5. Update the state machine so it places and then cancels an order, once it knows the book state. Use a fresh account with a small test balance. First run in replay mode against your old file and print the return values. Does it do what you expect? Then run it live, replay the session and make sure the replay actions match the ones in the log exactly. If not, fix the state machine and repeat until they do.
6. Congrats, this is a basic trading stack. Packets in → state machine → packets out. Push nondeterministic code to the edges. Make sure you can replay to fix the state machine.
7. Assuming you dumped steps 1-3 into a LLM, the networking code will be a mess. Learn what epoll is and and rewrite your networking code with a reactor pattern on top of epoll (or mio for convenience if using Rust + Mac). Add a timer event that fires every second, so your state machine can act even when nothing is happening. Use a nonblocking loop that polls the TCP connections, the action queue, and the timer, logging everything as an event.
8. Turn on kernel timestamping and attach the kernel ts from the last packet that completes each event alongside the system clock's timestamp. This indicates how long messages wait before your poll loop picks them up. This latency does not matter yet.
9. Likewise, the file writing and reading pattern was a bit of a hack. Wrap it in queue-like interface and add a comment to implement your own version of
https://github.com/OpenHFT/Chronicle-Queue
later for both the networking → sm and sm → networking directions. This latency also doesn't matter yet.
10. Modify the state machine to keep a bid and ask in the order book at a spread around the midprice. Configure a max inventory. When you have inventory, skew your orders on the side you were agressed to increase the distance from midprice by `spread * x / (1 - x)`, and on the other side decrease the distance from midprice by `spread * x`, where x is abs fraction of max inventory you hold.

11. The state machine code is probably getting VERY frustrating. The answer is MORE state machines. Factor out 3 smaller state machines. One to consume the inputs and update state (books, your orders, balances, etc). A second to take market state and compute the bid/ask prices and sizes your strategy wants. Add a TODO here to go back and factor out nice helpers for expressing desired orders when you start making more strategy changes. A third to compare the orders the strategy wants with the actual state and compute an action to take. Note that "actual state" includes pending states. Replay should produce identical outputs after refactoring. If not, you changed something.
12. Integrate a second exchange, e.g. Lighter, until you can run the same naive market making strategy there. Then pick an instrument listed on both venues, not super competitive, but with enough volume that you'll get fills. Make your strategy place a maker order on Lighter at the price you can hedge as a taker on Hyperliquid and add a profit margin. Every time the maker order trades, send a IOC taker order on Hyperliquid to hedge your position. Keep the skew function from before, but only consider your Lighter balance for your inventory for the purpose of skew. Run it with a very small spread so it's likely to fill and fix any bugs. Make sure it's tracking state, hedging properly, etc. and that you can replay deterministically.
13. Be meticulous. Are you computing fees correctly? Are you updating your position as early as possible? Pick through the log files line by line. Read the exchange docs. Are you subscribing to all the data you should be? Is your account stream faster or slower than the public book data? Does it depend? What are you using to trigger your hedge order, and is there something faster? What happens if you subscribe to each feed on 5 different TCP connections, do they tend to arrive all at the same time? Split your strategy fills into good and bad ones. What do you see on the feed around each kind of fill? Have a LLM make a dashboard where you can step through the log file line by line and see books, your orders, etc. Are you canceling and replacing your orders for no reason, losing queue position? Are you placing your limit orders ahead of a bunch of empty levels? Is someone else changing their order every time you place? Make sure you can leave the strategy running for 24h, handle disconnects and exchange issues, and use anything that goes wrong as an opportunity to improve the system. Aim to fill ~tens to hundreds of times per hour.
14. Build a simulator. You have the logs from the real trading session. You have your deterministic state machine. Write code to simulate how your orders would have interacted with the book to answer the question "if I hedged as a maker at the top of the book instead of a taker, what would my pnl be"? Then run the strategy hedging as maker, take the session file, and run the simulation again. Update until the sim *approximately* tracks reality - it does not need to be perfect, and you could easily spend a month on just the sim. For now just make it good enough to answer your immediate questions. Use seeded randomness for things like order placement latency. Make sure the same seed + capture + strategy gives the same result every time. You should be able to answer questions like "how would my pnl look if my system/network rtt were much faster?" or "is it worth it to pay priority fees for faster taker order on Hyperliquid?" quantitatively. Make it a habit to reconcile trading with your simulator to maintain its accuracy and dig into anything that makes it diverge.
15. You now have a trading system that works for basis trading across exchanges and catches good fills when someone pays through, but no signal research process or fair price model. Read
https://arxiv.org/pdf/2108.09750
for signal ideas. If the paper is unintelligible, read the relevant chapters of Elements of Statistical Learning. Set up a 24h capture, copy it somewhere, and use it as a poor man's research environment for now. You should be able to use it to synthesize clean data and predict Hyperliquid midprice changes a couple seconds in the future with decent r^2. You should not be able to monetize this, so apply the same signal research process to improving your fills on your cross-venue trade.
As a rough guideline, the whole trading stack should be ~5-15k lines as described. The venue specific code might get large but the core state machines should be small.
If you reach this point, it will be abundantly clear to you -- especially if you've written the important pieces yourself and gone on side quests where your understanding is weak -- that there are a million directions you can improve the

system in, and you don't have time to do them all. The world is your oyster and you can build your own syllabus.